

MCP (Model Context Protocol)

MCP (Model Context Protocol) is an open-source standard for connecting AI applications to external systems.

Using MCP, AI applications like Claude or ChatGPT can connect to data sources (e.g. local files, databases), tools (e.g. search engines, calculators) and workflows (e.g. specialized prompts)—enabling them to access key information and perform tasks.



Core Components of MCP Architecture

Two main entities: Host and Server

- **Host:** The AI chatbot interface interacting with users. It receives user prompts and sends them to an underlying LLM (Large Language Model) such as OpenAI, Anthropic, or Gemini.
- **Server:** A tool capable of executing specific tasks, such as interacting with GitHub repositories, Slack channels, or Google Drive files.
- Communication between Host and Server is **never direct**.
 - A **Client** acts as an intermediary between the Host and each Server.
 - Each Client speaks the same MCP language as the Server, enabling seamless communication.
- **One-to-one relationship between Client and Server**
 - Each Client connects exclusively to one Server.
 - To communicate with multiple Servers like GitHub, Slack, and Google Drive, the Host requires separate Clients for each Server.

Analogy:

Host = Phone, Client = SIM card, Server = Telecom network (e.g., Airtel, Jio). Just as phones don't communicate directly with the network without SIM, Hosts communicate via Clients.



MCP Primitives

Servers offer three types of primitives (basic offerings) to the Host:

Primitive	Description	Example
Tools	Actions the AI can ask the Server to perform	Creating GitHub issues, pushing code
Resources	Static or semi-static structured data the AI can read	GitHub README file, database schema
Prompts	Predefined prompt templates or instructions helping shape how the AI behaves with the Server	Guidelines to create well-formed GitHub issues

Tools allow dynamic operations (e.g., list commits, create issues).

Resources provide static data for reference (e.g., documentation).

Prompts (Props) guide AI behavior; for example, structuring issue reports with expected format details.

Standard Operations on Primitives

For **Tools**:

- **tools/list**: Client queries Server for available tools.
- **tools/call**: Client calls a specific tool with arguments.

For **Resources**:

- **resources/list**: Lists available resources.
- **resources/read**: Reads content of a specific resource.
- **resources/subscribe and resources/unsubscribe**: Allow for notifications on resource changes.

For **Prompts**:

- **prompts/list**: Lists available prompt templates.

- **prompts/get:** Fetches a specific prompt template.



Layers Of MCP

1 . Data Layer: JSON RPC 2.0

MCP uses **JSON RPC 2.0** as the foundation of its data layer.

JSON RPC is a protocol combining **Remote Procedure Calls (RPC)** with the simplicity of **JSON** for encoding.

It enables clients to call functions on remote servers as if they were local.

JSON RPC handles:

- **Request messages:** specifying the method name, parameters, and an id to match responses.
- **Response messages:** returning results or errors linked by the id.
- **Notifications:** one-way messages requiring no response, used for events like resource updates.
- **Batching:** multiple requests can be sent together in one batch.

Example request structure:

```
{"jsonrpc":"2.0","method":"add","params":[2,3],"id":1}
```

Example response structure:

```
{"jsonrpc":"2.0","result":5,"id":1}{"jsonrpc":"2.0","result":5,"id":1}}
```

JSON RPC supports **error codes** for failed calls, e.g., method not found.



2 . Transport Layer

The transport layer defines how JSON RPC messages travel between Client and Server. MCP supports two types of transport based on the nature of the Server:

Server Type	Transport Mechanism	Description
Local Server	STDIO (Standard Input/Output)	Server runs as a subprocess on the same machine; communication through process I/O streams.
Remote Server	HTTP + SSE (Server-Sent Events)	Server runs remotely; uses HTTP POST requests carrying JSON RPC payloads with SSE for streaming updates.

Local Server Communication:

- Host launches the Server as a subprocess.
- Exchanges JSON RPC messages via standard input/output streams.
- Benefits:
 - Very fast, secure (no network ports), simple to implement.

Remote Server Communication:

- Host sends HTTP POST requests containing JSON RPC messages.
- Uses HTTP authentication methods as needed.

SSE (Server-Sent Events) allows server to stream incremental updates to the client over a single open connection.

Ideal for:

- Long-running tasks.
- Incremental progress notifications in AI workflows.



Key Advantages of MCP Architecture

Decoupling: Separation of concerns between Host and multiple Servers. Servers and Clients operate independently without knowing about each other's communication.

Scalability: Host can connect to many Servers by adding corresponding Clients; supports parallel operations.

Flexibility: JSON RPC is **transport-agnostic**, enabling multiple transport mechanisms without changing the protocol.

Extensibility: Supports batching, streaming, notifications, and error handling natively.

Security and Performance: Local communication via STDIO is lightweight and secure; remote communication leverages HTTP's widespread features and security.